

MATNETS: Matrix-Neuron Neural Networks

A Research Library for Algebraically-Rich Deep Learning in JAX

[Project Report](#) · [Repository](#) · [Documentation](#) · [PyPI](#)

***TL;DR** — MATNETS replaces the scalar neuron with an $n \times n$ matrix neuron, turning every weight into a matrix-product operator and every activation into a structural transform. Built entirely on JAX primitives; composes with `jit`, `vmap`, and `grad` out of the box. Achieves **~98.5% MNIST accuracy at one-third the parameters** of a standard CNN.*

Highlights at a Glance

	MATNETS
Core idea	Every neuron is an $n \times n$ matrix, not a scalar
Computation	Full matrix multiplication (not element-wise)
Backend	Pure JAX — no PyTorch/TF dependency
Transforms	Fully compatible with <code>jit</code> , <code>vmap</code> , <code>grad</code> , <code>scan</code>
Architectures	Dense · Conv1D/2D · RNN · LSTM · GRU · Attention
MNIST result	~98.5% accuracy at ~0.33× the parameters of StdCNN
Install	<code>pip install matnets</code>

Installation

```
pip install matnets
```

Requires Python ≥ 3.10 and JAX (CPU or GPU). For GPU support, install the appropriate JAX build first:

```
# CPU
pip install jax

# CUDA 12 (GPU)
pip install "jax[cuda12]"

# Then install MATNETS
pip install matnets
```

Overview

MATNETS is an experimental JAX library that proposes a fundamental departure from the conventional neural network neuron model. Rather than treating each neuron as a scalar activation, MATNETS endows every neuron with an **$n \times n$ matrix**. This seemingly simple shift in representation opens up an entirely new algebraic regime for learning, where the basic unit of computation carries internal structure — orientation, volume, and linear map semantics — that a scalar cannot encode.

The Core Idea: Matrix-Neurons

In a standard dense layer, a neuron holds one number. In MATNETS, a neuron holds an $n \times n$ matrix. A layer of p input neurons and q output neurons therefore works with tensors shaped (p, n, n) and (q, n, n) respectively, and the weight tensor is (q, p, n, n) — a four-dimensional object where every (input neuron, output neuron) pair has an $n \times n$ matrix of weights. The bias is similarly matrix-valued, shaped (q, n, n) .

The dense primitive computes the following contraction:

$$\text{output} = \text{einsum}(\text{"qpak,pkc} \rightarrow \text{qac"}, W, x) + B$$

This is not Hadamard (element-wise) multiplication — it is full matrix multiplication along the inner indices, meaning each output matrix-neuron is a sum of matrix products of each input neuron's matrix with the corresponding weight matrix, plus a matrix bias. This distinguishes MATNETS from architectures that merely process matrices element-wise; the matrix product semantics are preserved throughout.

JAX Integration

MATNETS is designed as a set of pure JAX functions, not as a framework. This means the entire library composes seamlessly with JAX's functional transform ecosystem with no special handling required:

- `jax.jit` compiles the einsum-based dense kernel and all other primitives to XLA, producing efficient hardware-specific code.
- `jax.vmap` batches any forward function over a batch or token axis without changing the layer definitions — `vmap` adds the batch dimension transparently around the existing matrix-neuron shapes.
- `jax.grad` differentiates through all matrix operations, including the determinant-based activations and pooling, since JAX's automatic differentiation handles these algebraic expressions natively.
- `jax.lax.scan` powers the recurrent layers: each step is a closure over matrix-valued carry (H, C for LSTM), and `scan` unrolls the sequence in a compiled, memory-efficient manner.

Parameters are stored as `MatrixParams`, a named container for W and B that is registered as a JAX pytree. This means parameter trees work directly with `jit`, `vmap`, and `grad` without any custom rules — the standard JAX machinery sees `MatrixParams` just as it would a dict of arrays.

The library deliberately has no dependency on PyTorch or TensorFlow, requiring only JAX and its numpy interface. This keeps the stack minimal and the interop story clean.

Experimental Results: MNIST Benchmark

The attached plot compares four architectures on MNIST — **MatCNN**, **MatCNN0** (a variant without bias initialization warmup), **CapsNet**, and **StdCNN** — across 10 epochs, with the key constraint that the matrix-neuron models use roughly **one-third the parameters** of the standard CNN baseline.

Key observations:

- **MatCNN** reaches ~98% test accuracy by epoch 3 and converges smoothly to ~98.5% by epoch 10, closely tracking the standard CNN despite the parameter reduction.
- **MatCNN0** starts significantly lower (epoch 1 accuracy ~82%, training loss ~1.5), suggesting sensitivity to initialization, but recovers strongly by epoch 3 and converges on par with MatCNN.
- **CapsNet** achieves the best final accuracy (~99%), which is consistent with its routing mechanism's inductive bias for part-whole relationships.
- **StdCNN** performs solidly but does not outperform CapsNet, and closely matches MatCNN.

The central result is that **matrix-neuron CNNs match scalar CNNs at one-third the parameter count on MNIST**, suggesting the richer algebraic structure of matrix neurons packs more information per parameter. The initialization sensitivity of MatCNN0 also points to an interesting open question about how weight initialization should be adapted for the matrix product regime.

Open Research Directions

MATNETS is an early-stage research library, and the results so far open more questions than they close:

- **Determinant-based vs. element-wise activations at scale** — it remains unclear which regime dominates on harder tasks beyond MNIST.
 - **Recurrent matrix-neuron dynamics** — matrix-valued LSTM gates carry far more state per neuron; how this interacts with long-range memory is unexplored.
 - **Matrix attention at scale** — Frobenius-scored attention over matrix tokens is architecturally novel and has not been benchmarked against transformer baselines.
 - **Initialization theory** — Glorot-uniform applied to 4D weight tensors may not be optimal; the $n \times n$ matrix product semantics likely demand a rethinking of variance propagation.
 - **Computational cost vs. representational gain** — matrix neurons have $O(n^2)$ more parameters per neuron; the empirical question is how much more expressive this is relative to simply widening a scalar network.
-

Summary

MATNETS re-imagines the neuron as a matrix, making matrix multiplication — not scalar weighted addition — the fundamental operation of each layer. The library is built entirely on JAX's functional primitives, composes transparently with all of JAX's transforms, and extends the matrix-neuron idea to dense, convolutional, recurrent, and attention architectures. Initial experiments on MNIST show competitive accuracy with significantly fewer parameters, establishing proof of concept. The theoretical and empirical territory that follows — initialization, scaling, task generalization, the geometric semantics of determinant-based activations — remains wide open.

Repository: github.com/dsainvg/MATNETS | Documentation: dev.dsainvg.me/MATNETS | PyPI: [matnets](https://pypi.org/project/matnets)